

Intelligenza Artificiale

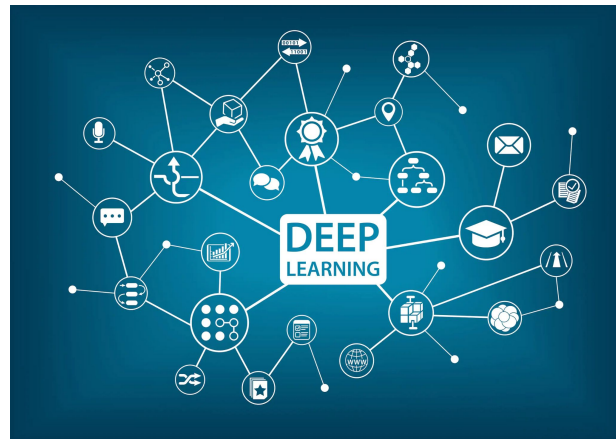
“Deep learning (part 1)”

Nicola Bellotto

A.A. 2025/2026

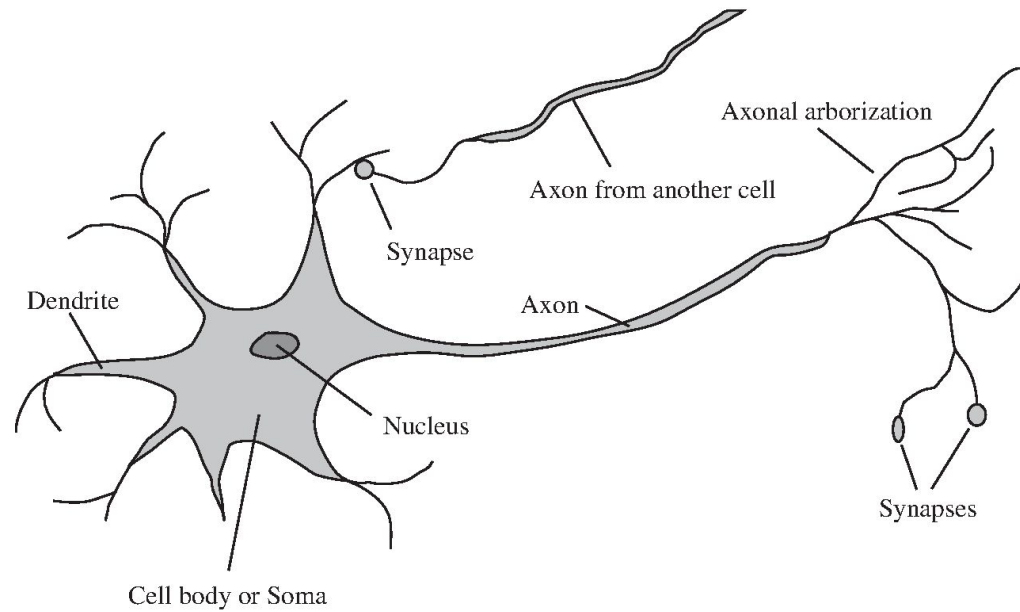
Lecture outline

- Artificial neural networks
- Activation functions
- Multilayer networks
- Deep neural networks
- Convolutional neural networks

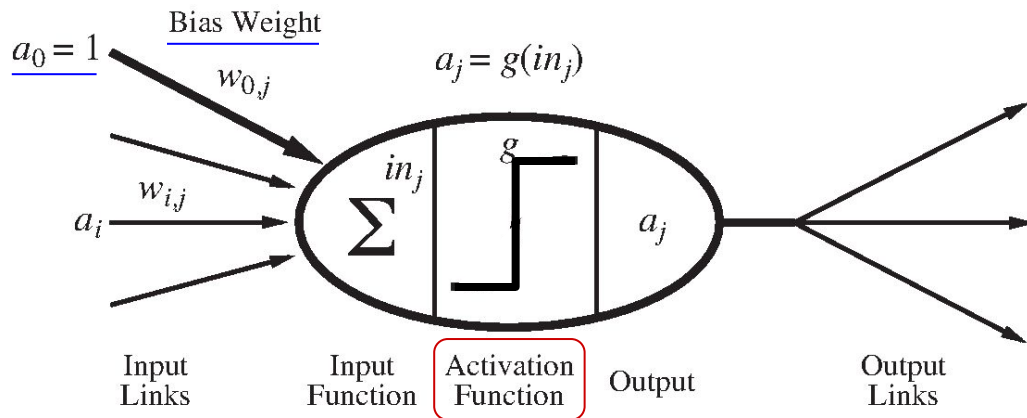


Slides partly based on Russell & Norvig instructors material <http://aima.cs.berkeley.edu/instructors.html>

Neuron



Perceptron

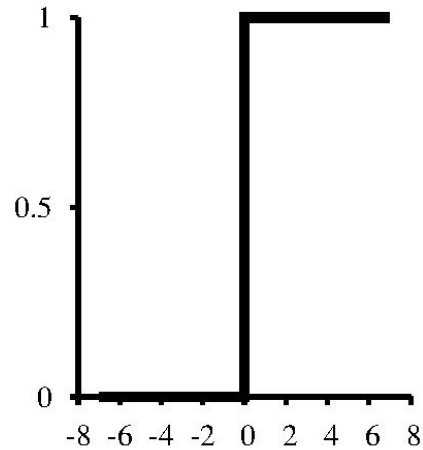


$$a_j = g\left(\sum_{i=0}^n w_{i,j} a_i\right)$$

Threshold activation function

$$g(\cdot)$$

Threshold

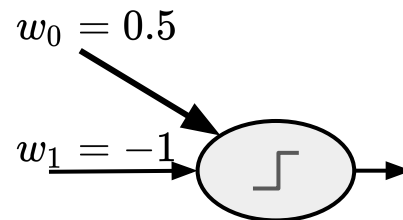
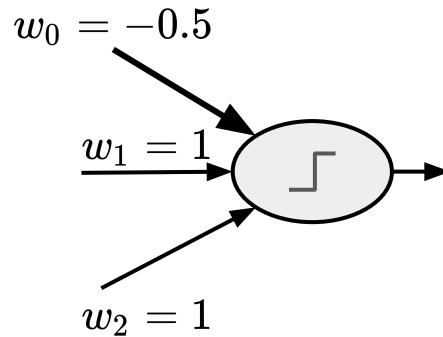
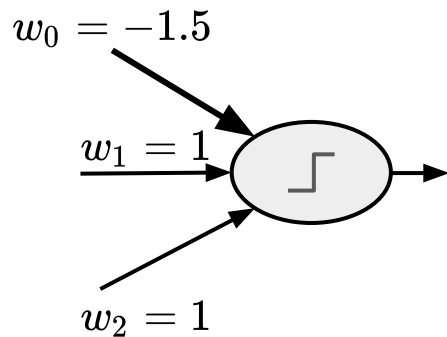


Example: Boolean functions

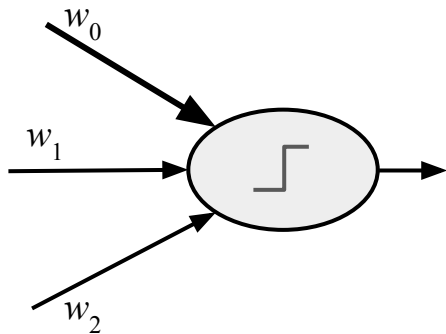
x_1	x_2	<i>AND</i>
0	0	0
0	1	0
1	0	0
1	1	1

x_1	x_2	<i>OR</i>
0	0	0
0	1	1
1	0	1
1	1	1

x_1	<i>NOT</i>
0	1
1	0

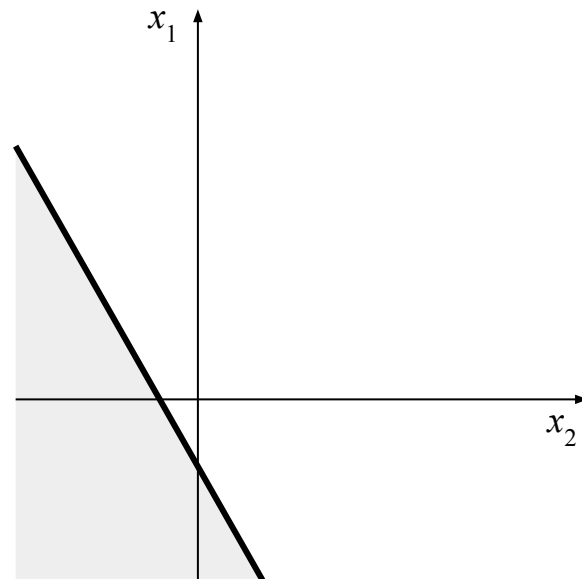


Threshold perceptron



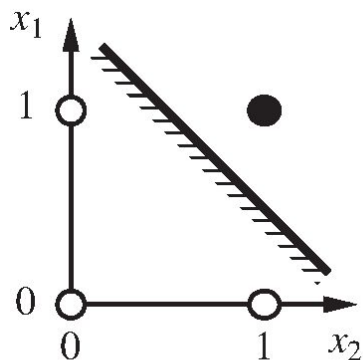
$$w_0 + w_1 x_1 + w_2 x_2 > 0$$

$$x_1 > -\frac{w_2}{w_1} x_2 - \frac{w_0}{w_1}$$

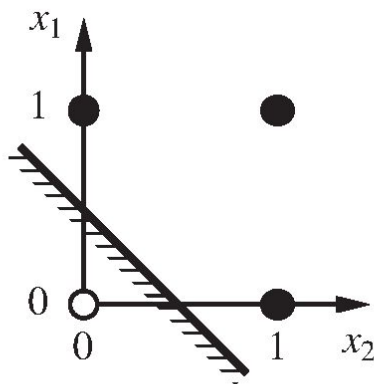


Linear separability

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

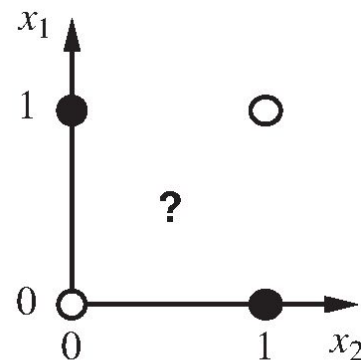


(a) x_1 **and** x_2



(b) x_1 **or** x_2

Linearly separable



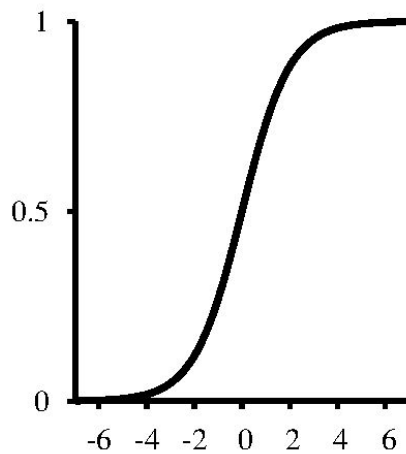
(c) x_1 **xor** x_2

Not linearly separable

Sigmoid activation function

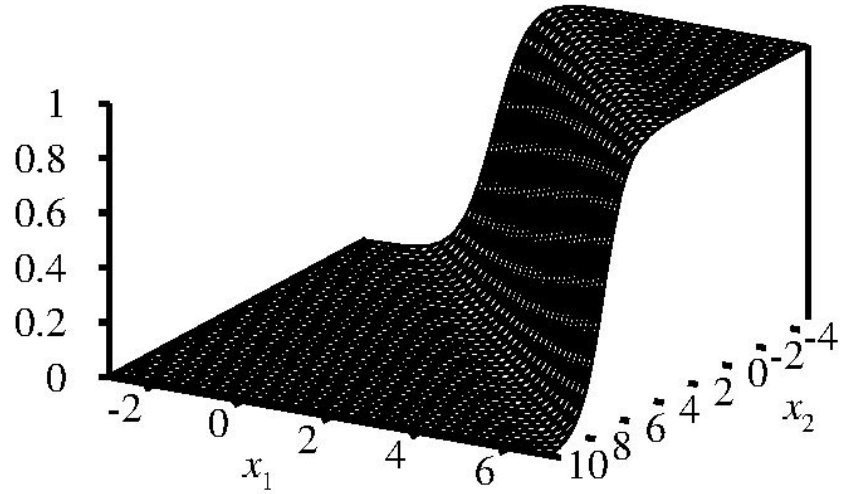
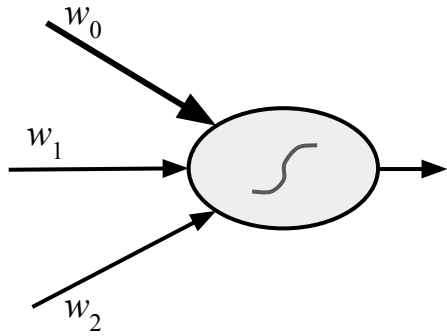
$$g(\cdot)$$

Sigmoid (or Logistic)



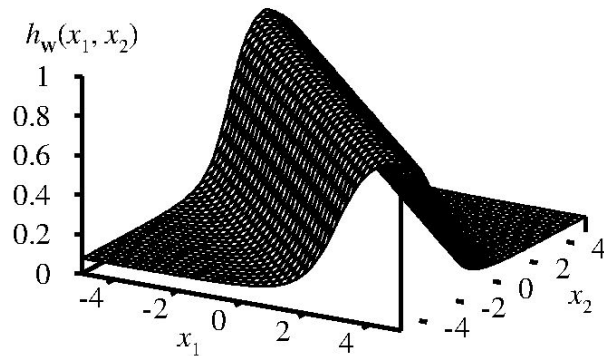
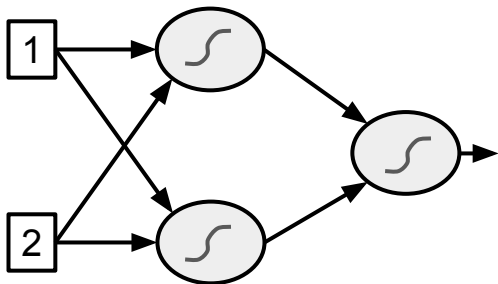
$$1/(1 + e^{-x})$$

Artificial neuron with sigmoid activation



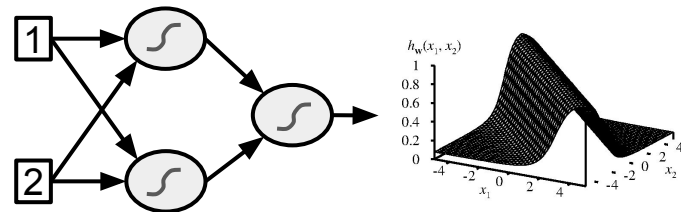
Combination of neurons

- The combination of 2 (hidden) neurons with soft thresholds into 1 (output) neuron produces a “ridge”

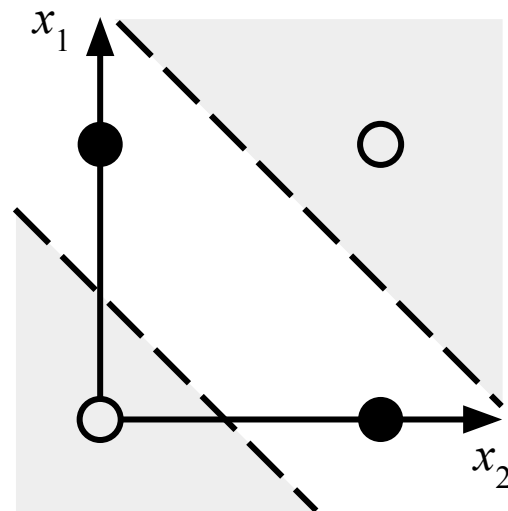


Example: XOR function

- The XOR function can be implemented by 3 neurons (2 hidden + 1 output)
- In theory we could use “hard” thresholds, but the differentiability of the “soft” sigmoid facilitates the training
- The same concept can be extended to deal with non-linearly separable functions in n dimensions

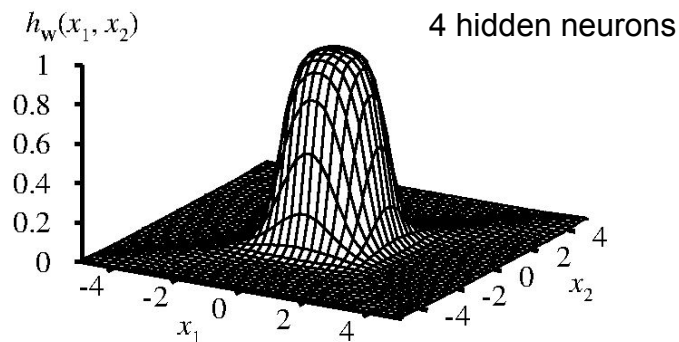
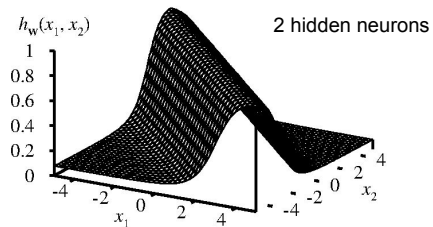
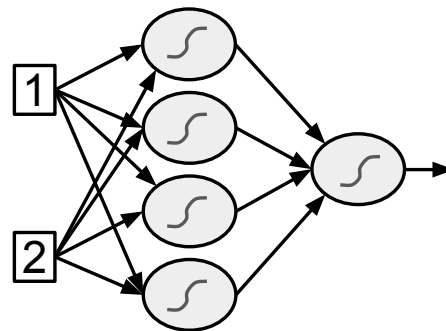


x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

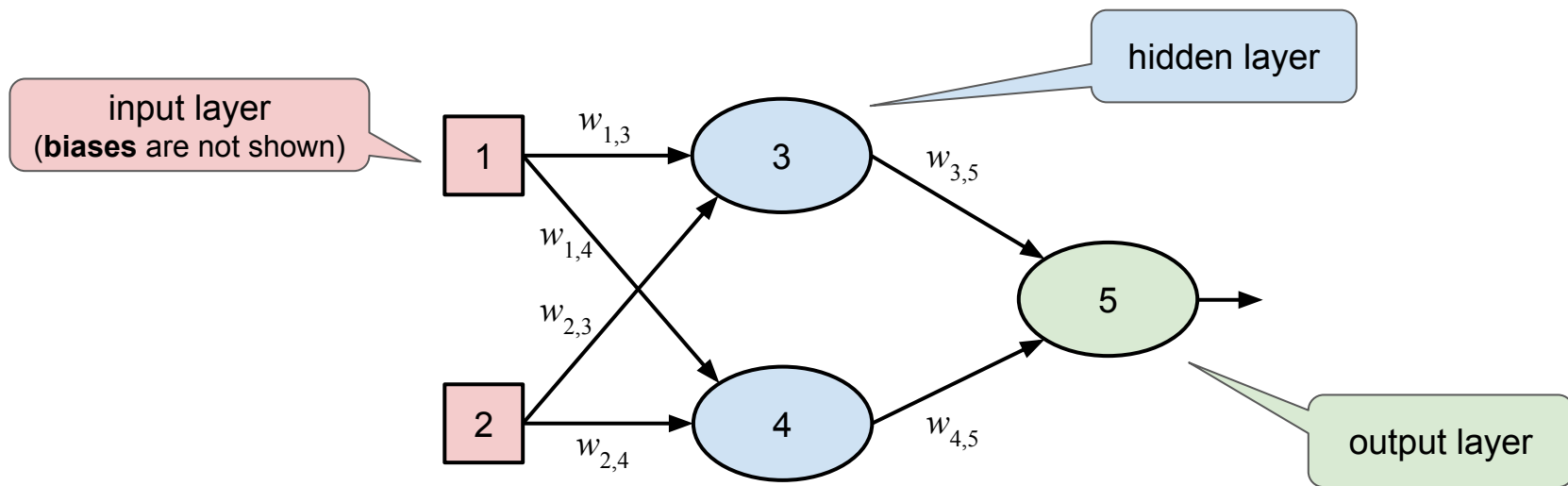


Universal approximation

- The combination of 2 ridges using 4 hidden neurons produces a “bump”
- Any continuous functions can be approximated with just two layers
(**universal approximation theorem**)



Multilayer neural network

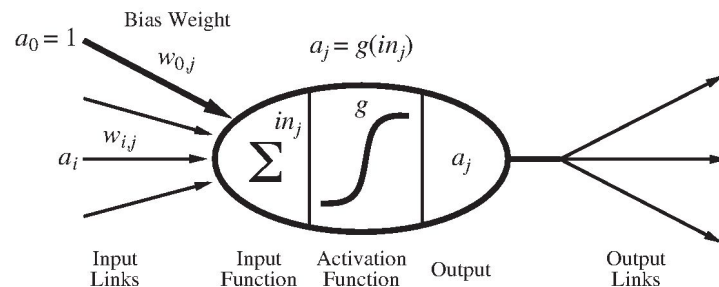


$$\begin{aligned} a_5 &= g(w_{0,5} + w_{3,5}a_3 + w_{4,5}a_4) \\ &= g(w_{0,5} + w_{3,5}g(w_{0,3} + w_{1,3}x_1 + w_{2,3}x_2) + w_{4,5}g(w_{0,4} + w_{1,4}x_1 + w_{2,4}x_2)) \\ &= h_{\mathbf{w}}(\mathbf{x}) \end{aligned}$$

Networks as complex functions

- Each node (neuron) within a network is called a **unit**
 - calculates the weighted sum of the inputs from predecessor nodes
 - applies a nonlinear function to produce its output

$$a_j = g_j\left(\sum_i w_{i,j}a_i\right) \equiv g_j(in_j)$$



- g_j is a nonlinear activation function (otherwise the combination would still be linear)
- a_j denotes the output of unit j
- $w_{i,j}$ is the weight attached to the link from unit i to unit j

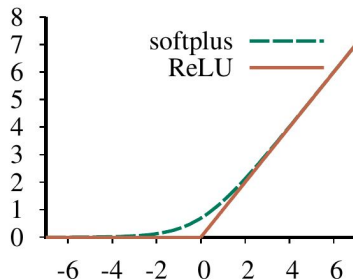
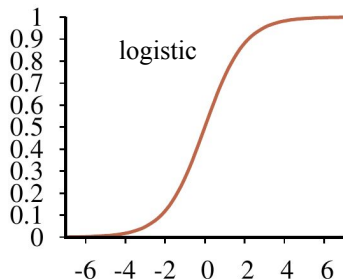
Common activation functions

- **Sigmoid** (or **Logistic**)

$$\sigma(x) = 1/(1 + e^{-x})$$

- **Rectifier Linear Unit (ReLU)**

$$\text{ReLU}(x) = \max(0, x)$$

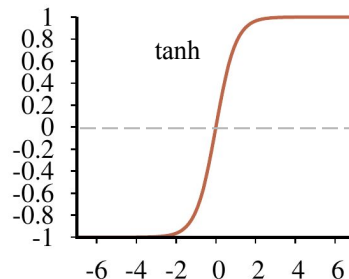


- **Softplus** (smooth version of ReLU)

$$\text{softplus}(x) = \log(1 + e^x)$$

- **Tanh** (scaled and shifted sigmoid in (-1, +1))

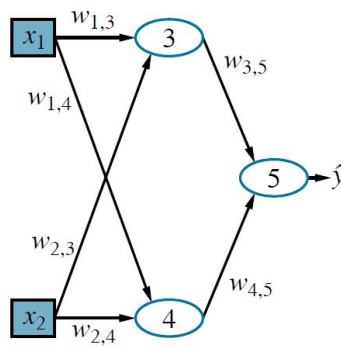
$$\tanh(x) = \frac{e^{2x} - 1}{e^{2x} + 1}$$



Neural network representations

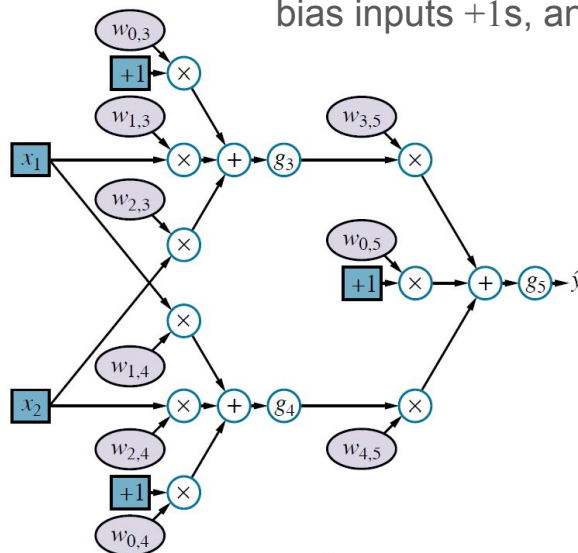
Two examples of common representations:

(a) network with two inputs, one hidden layer of two units, one output (no bias shown)



(a)

(b) same neural network represented by a full **computation graph**, including operations, bias inputs +1s, and their weights



(b)

Deep neural networks

- The number of hidden layer makes the depth of the neural network

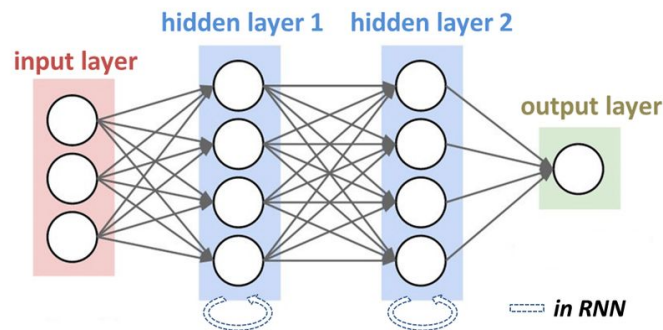
- Two basic classes are

- **Feedforward network**

- connections only in one direction (input $\rightarrow$ output)
- directed acyclic graph with input and output nodes
 - i.e. no loops

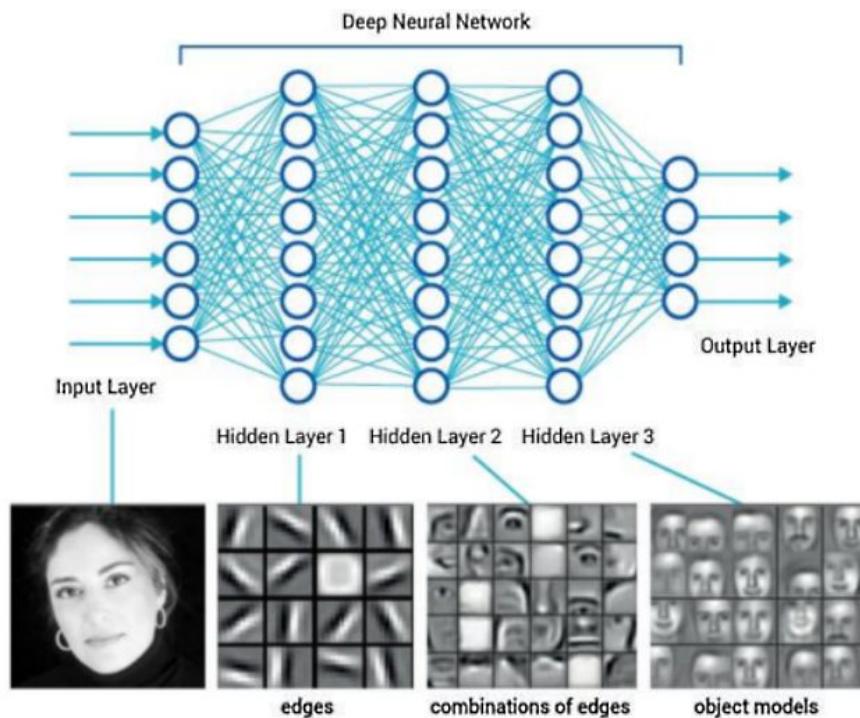
- **Recurrent network (RNN)**

- its intermediate or final outputs back into its own inputs
- network form a dynamical system that has **internal state** or **memory**



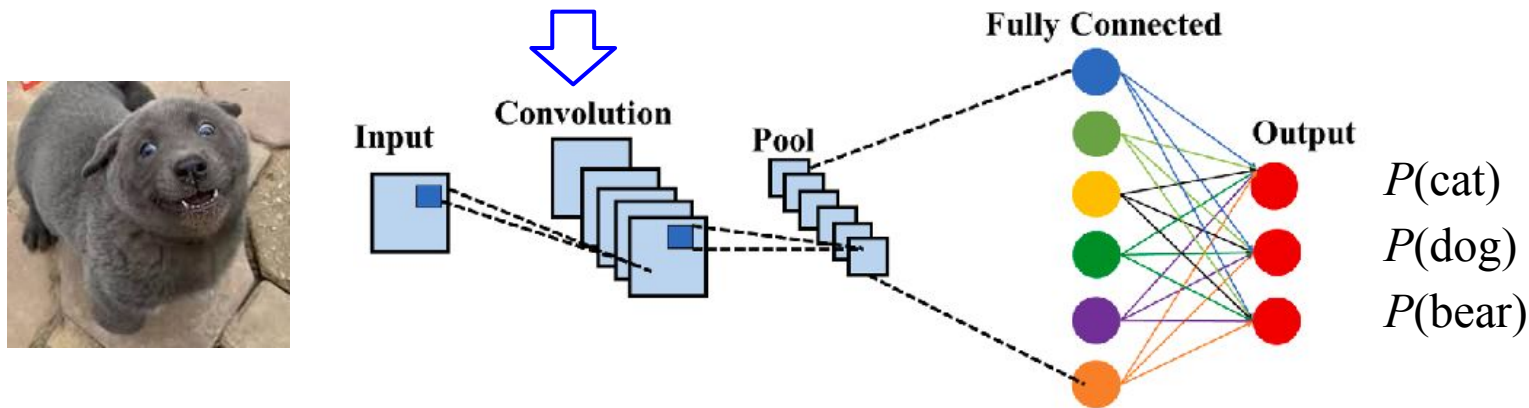
Hidden layer representations

- Hidden layers usually capture some inner representations of the input layer, e.g.



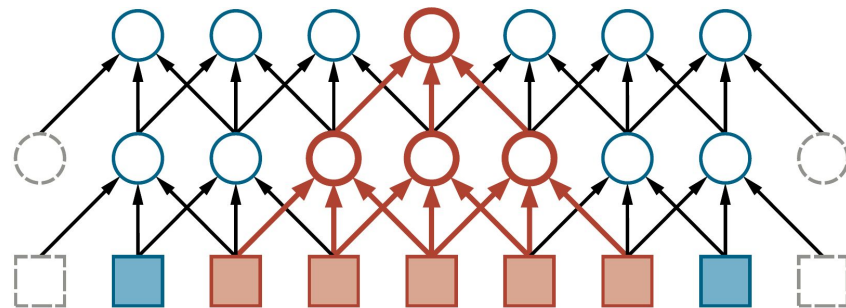
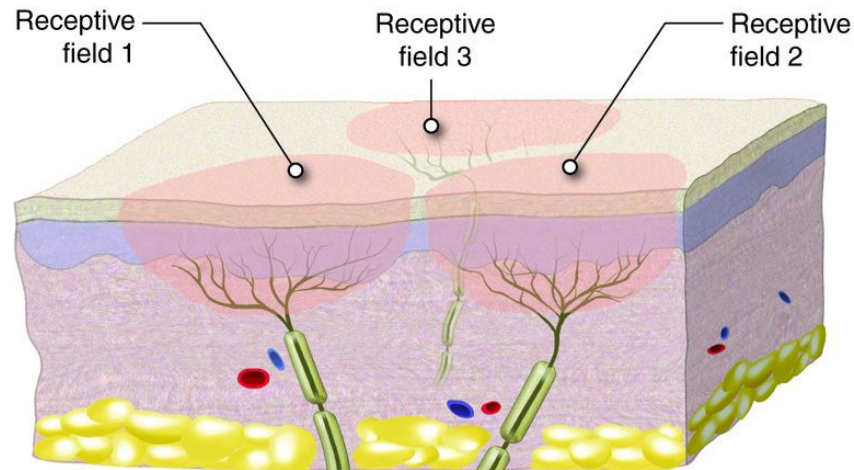
Convolutional Neural Networks (CNNs)

- **Convolutional neural network (CNN)** is one of the first and most popular deep neural networks
- Name comes from (hidden) **convolutional layers** that contain spatially local connections, often followed by *pooling layers* and further (sub)networks
 - i.e. each convolution layer is connected only to a sub-region of the input image



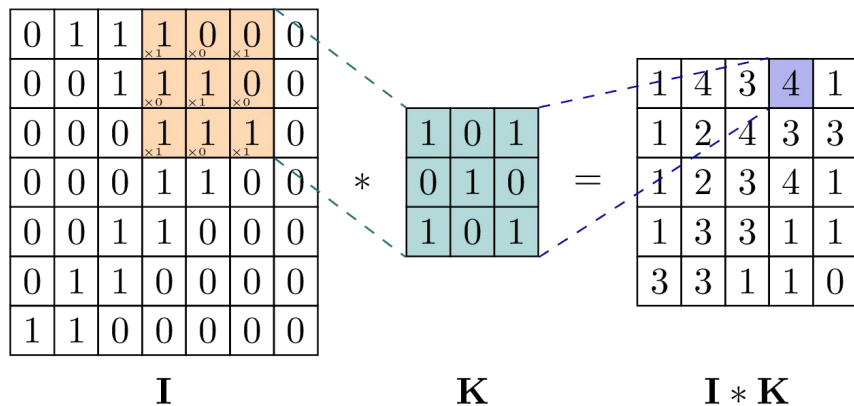
Receptive field

- CNNs were inspired by models of the visual cortex where the **receptive field** of a neuron can affect its activation
- In a CNN, the receptive field of a unit in the first hidden layer is small
- Generally speaking, the deeper the unit, the larger the receptive field
 - E.g. shown in red is the receptive field of a unit in the second hidden layer



Convolution operation

- Main ingredients
 - **Kernel**: pattern **K** of weights (learned during training)
 - **Convolution**: process of applying kernel **K** to inputs **I** (e.g. pixels of an image)



Kernel

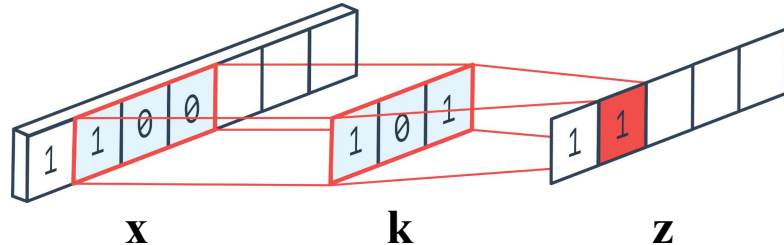
$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$



Convolution layer

- For simplicity, let's consider an input vector $\mathbf{x}$ of size n , corresponding to n pixels in a one-dimensional image, and vector kernel $\mathbf{k}$ of size l
- Convolution operation, $\mathbf{z} = \mathbf{x} * \mathbf{k}$
 - Dot product of kernel $\mathbf{k}$ and snippet of $\mathbf{x}$ centered in x_i

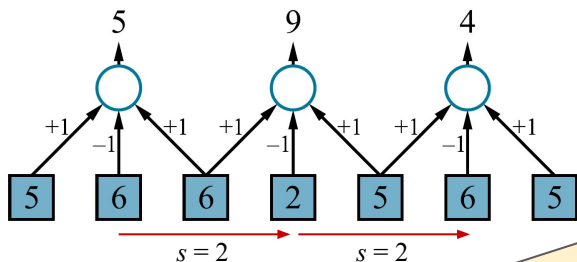
$$z_i = \sum_{j=1}^l k_j x_{j+i-(l+1)/2}$$



- The results usually go through a nonlinear activation function before the next hidden layer

Stride

- Convolution can be viewed as a matrix multiplication
- Kernels centers are separated by distance called **stride** s
 - E.g. a one-dimensional convolution operation with a kernel of size $l = 3$ and a stride $s = 2$

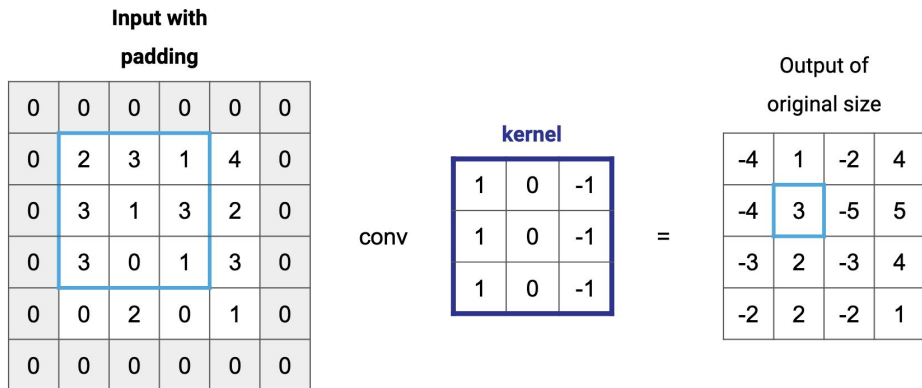


$$\begin{pmatrix} +1 & -1 & +1 & 0 & 0 & 0 & 0 \\ 0 & 0 & +1 & -1 & +1 & 0 & 0 \\ 0 & 0 & 0 & 0 & +1 & -1 & +1 \end{pmatrix} \begin{pmatrix} 5 \\ 6 \\ 6 \\ 2 \\ 5 \\ 6 \\ 5 \end{pmatrix} = \begin{pmatrix} 5 \\ 9 \\ 4 \end{pmatrix}$$

In the weight matrix, the kernel $[+1, -1, +1]$ appears in each row, shifted according to the stride

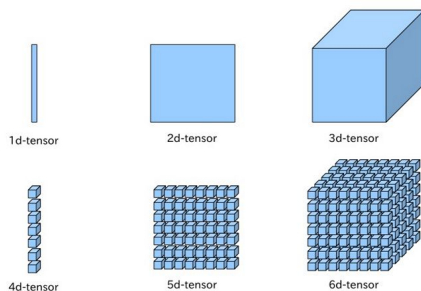
Padding

- Convolution stops at the edges of the image
- **Padding** the input with extra pixels allows the kernel to be applied exactly $\lfloor n/s \rfloor$ times
 - e.g. in the padded image below the kernel can be applied 4 x 4 times (with $s = 1$) instead of only 2 x 2 with the original image

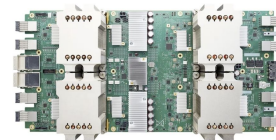


Tensor operations in CNNs

- **Tensors** are simply **multidimensional arrays** of any dimension
 - Vectors / matrices are 1-dimensional and 2-dimensional special cases of tensors
- A way to keep track of the data “shape” as it progresses through the network’s layers

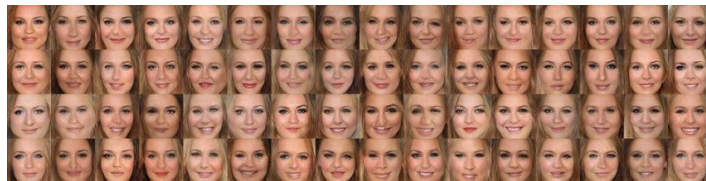


- Given a network’s description as a sequence of tensor operations, deep learning software package can generate highly optimized code
- They run on **GPUs** (graphics processing units) or **TPUs** (tensor processing units) with a high degree of parallelism



Example with tensors

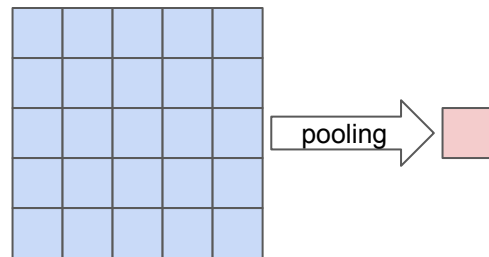
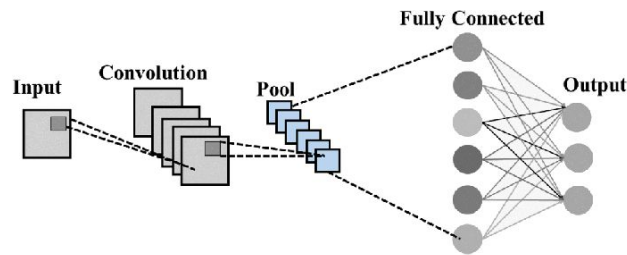
- Training input: 64 RGB images $256 \times 256 \times 3$
⇒ input tensor $256 \times 256 \times 3 \times 64$



- 96 kernels $5 \times 5 \times 3$ (each one processing RGB subimage of 5×5 pixels)
- stride $s = 2$ (only on x-y dimensions, not RGB, with padding 2)
⇒ output after applying 1 kernel to 1 image: $(n/s) \times (n/s) = 128 \times 128$
- ⇒ final output tensor $128 \times 128 \times 96 \times 64$ (also called **feature map** with 96 channels)

CNN pooling

- A pooling layer usually summarizes a set of adjacent units from the preceding layer with a single value
 - a special case of convolution with fixed weights (not learned)
- Common forms of pooling
 - **Average-pooling**: computes the average value of its s inputs
 - Coarsen the resolution of the image, downsample by a factor of s
 - **Max-pooling**: computes the maximum value of its s inputs
 - Can be used for downsampling
- Often used for image classification to reduce size of hidden layers



Conclusions

- Suggested reading
 - Chapter 22, sec. 22.1–22.3 of Russell & Norvig
- Next lecture
Deep learning (part 2)
- Questions?

